

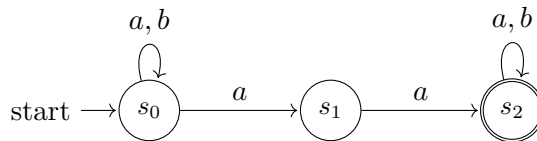
Problem Set 3: Solutions

Automating Lexical Analysis (Lecture 3)

Compiler Construction Module

1 NFAs, nondeterminism, and the DFA blow-up

The NFA over $\{a, b\}$ for “contains aa”: $s_0 \xrightarrow{a,b} s_0$, $s_0 \xrightarrow{a} s_1 \xrightarrow{a} s_2$, $s_2 \xrightarrow{a,b} s_2$, accept s_2 .



Problem: 1(a): an accepting path for baab

Solution

$$s_0 \xrightarrow{b} s_0 \xrightarrow{a} s_1 \xrightarrow{a} s_2 \xrightarrow{b} s_2 \in F.$$

Choices: on the first **b** there is only the self-loop. On the first **a** the machine has two options (stay at s_0 or move to s_1); the accepting path *guesses* s_1 , betting that another **a** follows. On the second **a** it moves $s_1 \rightarrow s_2$ (the only transition). On the final **b** it stays at s_2 . Since one path ends in an accept state, **baab** is accepted.

Problem: 1(b): why abab is rejected

Solution

Track the *set* of states reachable after each symbol, which covers every possible sequence of choices at once:

$$\{s_0\} \xrightarrow{a} \{s_0, s_1\} \xrightarrow{b} \{s_0\} \xrightarrow{a} \{s_0, s_1\} \xrightarrow{b} \{s_0\}.$$

After **ab** the set is back to $\{s_0\}$ because s_1 has no **b**-transition (the guess “an **aa** starts here” dies). The same happens again. The final set $\{s_0\}$ contains no accept state, so no choice sequence reaches s_2 : rejected. Intuitively, reaching s_2 requires two *consecutive* **a**’s, and **abab** has none.

Problem: 1(c): subset construction

Solution

There are no ε -edges, so closures are trivial. Start $A = \{s_0\}$.

- $A = \{s_0\}$: on a : $s_0 \rightarrow s_0, s_1$, giving $B = \{s_0, s_1\}$; on b : $s_0 \rightarrow s_0$, giving A .
- $B = \{s_0, s_1\}$: on a : $s_0 \rightarrow s_0, s_1$ and $s_1 \rightarrow s_2$, giving $C = \{s_0, s_1, s_2\}$; on b : only $s_0 \rightarrow s_0$, giving A .
- $C = \{s_0, s_1, s_2\}$ (accepting, contains s_2): on a : $s_0 \rightarrow s_0, s_1$, $s_1 \rightarrow s_2$, $s_2 \rightarrow s_2$, giving C ; on b : $s_0 \rightarrow s_0$, $s_2 \rightarrow s_2$, giving $D = \{s_0, s_2\}$.
- $D = \{s_0, s_2\}$ (accepting): on a : $s_0 \rightarrow s_0, s_1$, $s_2 \rightarrow s_2$, giving C ; on b : $s_0 \rightarrow s_0$, $s_2 \rightarrow s_2$, giving D .

No new sets. Transition table:

δ_D	a	b		
A (start)	B	A	$A = \{s_0\}$	$C = \{s_0, s_1, s_2\}$
B	C	A	$B = \{s_0, s_1\}$	$D = \{s_0, s_2\}$
C (accept)	C	D		
D (accept)	C	D		

Problem: 1(d): how many states, and what it illustrates

Solution

Four DFA states from a three-state NFA. The bound from subset construction is $2^3 = 8$; only the four *reachable* subsets appear, which is typical: the exponential worst case is real (there are languages, such as “the n -th symbol from the end is a ,” where every DFA needs 2^n states) but most practical NFAs blow up far less. The DFA is also not minimal: C and D are equivalent (both accept every continuation, since once aa has been seen nothing can undo it), so minimisation merges them and the minimal DFA has three states: “no a pending,” “one a pending,” “done.”

2 Thompson’s construction for $(a|b)^*c$

Problem: 2(a): rules applied in order, with state counts

Solution

Parse first: $(a | b)^*c = E_1 \cdot E_2$ with $E_1 = E_0^*$, $E_0 = a | b$, $E_2 = c$. Build inside-out:

rule applied	new states	running total
symbol a	+2	2
symbol b	+2	4
union $a \mid b$ (new start, new accept)	+2	6
star $(a \mid b)^*$ (new start, new accept, skip edge, loop-back edge)	+2	8
symbol c	+2	10
concatenation $(a \mid b)^* \cdot c$ (one ε -edge, no states)	+0	10

Problem: 2(b): total

Solution

Ten states.

Problem: 2(c): why the same total as $(0 \mid 1)^* 1$

Solution

Thompson's construction never looks at *which* symbols appear, only at the *shape* of the expression: every symbol leaf costs 2 states, every union costs 2, every star costs 2, every concatenation costs 0. Both expressions have the same syntax tree: a concatenation whose left child is a star of a union of two symbols and whose right child is one symbol. Rule by rule: symbol (0 vs a) +2, symbol (1 vs b) +2, union +2, star +2, symbol (1 vs c) +2, concatenation +0. Three leaves and two operators give $2 \times 5 = 10$ in both cases. Renaming $0 \rightarrow a$, $1 \rightarrow b$, and the trailing $1 \rightarrow c$ turns one NFA into the other.

3 Subset construction with ε -transitions

The NFA for “ x or xy ”: $r_0 \xrightarrow{\varepsilon} r_1 \xrightarrow{x} r_2$ (accept), $r_0 \xrightarrow{\varepsilon} r_3 \xrightarrow{x} r_4 \xrightarrow{y} r_5$ (accept).

Problem: 3(a): ε -closure of r_0 , then the transition on x

Solution

$\varepsilon\text{-closure}(r_0) = \{r_0, r_1, r_3\}$: r_0 itself plus the two states reachable by ε -edges; neither r_1 nor r_3 has further ε -edges. This is the DFA start state Q_0 .

On x from Q_0 : $r_1 \xrightarrow{x} r_2$ and $r_3 \xrightarrow{x} r_4$; r_0 has no x -edge. Closure of $\{r_2, r_4\}$ adds nothing (no ε -edges leave them). So $\delta_D(Q_0, x) = Q_1 = \{r_2, r_4\}$, which is **accepting** because it contains r_2 : the input x alone is a valid

string.

Problem: 3(b): from Q_1 on y

Solution

$r_4 \xrightarrow{y} r_5$; r_2 has no y -edge. $\delta_D(Q_1, y) = Q_2 = \{r_5\}$, **accepting**: xy is in the language. The DFA has recorded “both branches were alive after x ; only the xy branch survived the y .”

Problem: 3(c): from Q_1 on a second x

Solution

Neither r_2 nor r_4 has an x -edge, so the set of reachable NFA states is empty: $\delta_D(Q_1, x) = \emptyset$, the error state. xx is not in the language, and the DFA says so by having nowhere to go. In a lexer this is exactly the “stuck” signal that ends read-ahead: the longest accepted prefix of $xx\dots$ is the first x .

4 DFA minimisation

DFA for “contains at least one 1”: v_0 start; $v_0 \xrightarrow{0} v_1$, $v_0 \xrightarrow{1} v_2$; $v_1 \xrightarrow{0} v_1$, $v_1 \xrightarrow{1} v_3$; v_2 and v_3 accepting and absorbing.

Problem: 4(a): the distinguishability table

Solution

Six pairs. **Rule 1** (one accepting, one not): (v_0, v_2) , (v_0, v_3) , (v_1, v_2) , (v_1, v_3) are marked with witness ε . Unmarked after Rule 1: (v_0, v_1) and (v_2, v_3) .

Rule 2, sweep 1.

- (v_0, v_1) : on 0, successors (v_1, v_1) : the same state, nothing to look up. On 1, successors (v_2, v_3) : unmarked. Nothing fires.
- (v_2, v_3) : on 0, successors (v_2, v_3) : itself, unmarked. On 1, successors (v_2, v_3) : same. Nothing fires.

Sweep 2. The DFA and the marks are unchanged, so the result is identical: nothing new. Stop.

	v_0	v_1	v_2
v_1	unmarked		
v_2	ε	ε	
v_3	ε	ε	unmarked

Problem: 4(b): equivalence classes

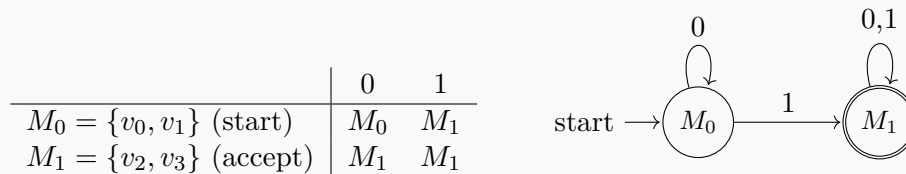
Solution

Two classes: $\{v_0, v_1\}$ (“no 1 seen yet”; v_1 merely remembers that some 0’s were read, which the language does not care about) and $\{v_2, v_3\}$ (“a 1 has been seen”; both accept everything from now on). Every string is accepted from v_0 iff it is accepted from v_1 (iff it contains a 1), and every string is accepted from both v_2 and v_3 .

Problem: 4(c): the minimised DFA

Solution

Merge each class into one state and relabel every transition:



Two states, the obvious minimum: a one-state DFA would accept everything or nothing.

5 Longest match and rule order

Rules: "class" $\rightarrow$ CLASS, and $[a-zA-Z][a-zA-Z0-9_]* \rightarrow$ ID.

Problem: 5(a): could classroom become CLASS + ID("room")?

Solution

Never. At the start of `classroom` the keyword rule matches 5 characters and the identifier rule matches all 9. Longest match picks the 9-character token, so the result is `ID("classroom")`, and the input position moves past the whole word. There is no later moment at which the lexer reconsiders and cuts the word in two: a lexer emits the longest token starting at the current position and moves on.

Problem: 5(b): input exactly class

Solution

Longest match alone does *not* resolve it: both rules match exactly 5 characters, and the combined DFA lands in a state tagged with both CLASS and ID. The tie is broken by **rule order**: the rule written first in the specification wins. In every sane spec the keyword rule precedes the identifier rule, so the token is CLASS. (If the identifier rule were listed first, `class` would lex as an identifier and the keyword would be unreachable.)

Problem: 5(c): when is rule order consulted?

Solution

Only when two or more rules match the *same, longest* prefix at the current position, so that length cannot decide between them.